

The RAKSHA Book

Everything about RAKSHA AI in one place: what we are building and why, how every part works, what the operator sees, where we stand today, and what the finished system should look like.

Status as of 5 October 2026 · Branch `claude/magical-mayer-gs2xno` · Last full test run: 873 passed, 0 failed

PART 0

Start here

1 How to read this book

Every chapter opens with a short summary and closes with key takeaways. Technical words are explained the first time they appear, in a purple box or with a dotted underline. Colour always means the same thing:

BLUE · IN PLAIN WORDS

The simple explanation, first.

GREEN · BUILT AND TESTED

Working today, with tests behind it.

AMBER · PARTIAL

Built, but needs the finale hardware or a team step.

RED · NOT BUILT / LIMITATION

Missing, or an honest weakness.

PURPLE · TERM EXPLAINED

Jargon, decoded where it first appears.

GREY · FOR ENGINEERS

Detail you can safely skip.

YELLOW DASHED · YOUR NOTES

Space at the end of chapters for your suggestions.

Two reading paths. A commander or reviewer can read Parts 0, I, II, IV, V and VIII and skip the grey boxes. An engineer should read everything, including Part III in full.

2 RAKSHA in one page

IN PLAIN WORDS

RAKSHA is a self-driving security mechanic for software. You give it a program's source code. It finds weaknesses, proves each one is real by making it happen in a safe box, writes a fix, and proves the fix works with five independent checks. It hands you a sealed receipt for every claim. It does all of this offline, with the network cable pulled, because Army networks are sealed.

- **Find:** fast checks that need no compiling, plus deep checks that run the code with thousands of test inputs.
- **Prove the weakness:** nothing is reported unless it can be reproduced. "No reproducer, no report."
- **Fix:** a ladder of fixers from cheapest to most expensive: known fix patterns, past fixes, then an AI model.
- **Prove the fix:** a five-check gate decides. The AI never decides anything.
- **Show it:** an offline console for operators and commanders, in English and Hindi, with signed reports a judge can re-check on their own laptop.
- **Remember:** every project gets an ID and a versioned, colour-coded history; recurring faults become proposed guidelines.

KEY TAKEAWAYS

- RAKSHA shows proof, not opinions.
- It works offline, on unfamiliar code, in several languages.
- Humans keep authority: rules of engagement decide what it may do on each system.

PART I

The problem and the aim

What the competition asks for, why it is hard, and what RAKSHA promises in return.

3 The competition

AI Kavach is a challenge to build working technology for real defence and national-security problems. The task for this track, in the organisers' words, is a cyber-reasoning system that autonomously finds a vulnerability, patches it, and proves the fix holds, pitched to run against customised infrastructure of the Indian Armed Forces.

TERMS EXPLAINED

Cyber-reasoning system (CRS): a program that reasons about the security of other programs by itself. **Vulnerability:** a weakness in software that someone could misuse. **Patch:** a change to the code that removes the weakness.

Autonomously: without a human in the loop during the find-fix-prove cycle.

Air-gapped: physically disconnected from any outside network.

How it is judged

STAGE	WHAT IS SCORED
Shortlisting (5 slides)	Resource utilisation · novelty of idea · how lightweight the solution is
Grand Finale (36-hour build)	Performance (bugs found and patched) · speed · precision (how often it is right) · functionality (the full loop, autonomously) · scalability (many targets and languages)

4 The problem

Finding and fixing weaknesses in software is slow, skilled work. Army software adds three hard conditions:

- **Unfamiliar code.** Custom systems no public tool has ever seen, in several programming languages.
- **Sealed networks.** No internet, so no cloud AI and no online databases at the point of use.
- **High stakes.** A wrong "fix" on a mission system can do more harm than the weakness. Every claim must be checkable.

5 Why today's AI tools fall short

- **They guess.** General AI models produce plausible fixes that are often wrong; published evaluations show frontier models failing a large share of real repair tasks.
- **They need the internet.** The strongest models run in the cloud, which a sealed network forbids.
- **They give no proof.** A static scanner says "this line might be risky". A commander cannot act on "might".

TERM EXPLAINED

Static scanner: a tool that reads code without running it and flags patterns. Fast, but it cannot show that a problem really happens.

6 Our aim and promise

IN PLAIN WORDS

Find real weaknesses in code nobody has seen before, fix them, and hand over evidence anyone can re-check, all offline, with humans in charge of what gets applied.

The promise rests on one design choice: the AI model is an intern, not the judge. It may suggest. Deterministic checks decide.

TERM EXPLAINED

Deterministic: gives the same answer every time for the same input. A check that cannot be talked out of its answer.

7 Scope

IN SCOPE

- Source code in C/C++, Python, Java, JavaScript/TypeScript, Go and Rust (deep checks)
- Any language for build-free checks (dependencies, secrets, crypto, configuration, API specs, binaries, git history)
- Proving, fixing and re-proving; signed evidence; reports for operators and commanders
- Offline operation on a single sealed machine

OUT OF SCOPE

- Attacking live systems or networks: RAKSHA only works on code handed to it, inside its own sandbox
- Applying fixes to production by itself on critical systems (humans approve)
- Claiming a system is "secure": RAKSHA reports what it found, never the absence of problems

8 Ground rules we never break

1. **No reproducer, no report.** Every reported weakness can be made to happen again.
2. **The model never decides.** It proposes; the five-check gate decides.
3. **Presence, never absence.** RAKSHA says what it found. It never says "secure".

4. **Nothing leaves the box.** No code, finding or report goes over a network. This is measured, not promised.
5. **Every number is measured or empty.** If something was not measured, the screen shows a dash, never a made-up figure.
6. **Untrusted code runs only in the sandbox.**
7. **Zero false alarms on clean controls.** Deliberately clean code must produce no findings, checked after every change.

TERM EXPLAINED

Reproducer: the exact input that makes the weakness happen, kept so anyone can replay it. **Sandbox:** a sealed box (a container with no network) where untrusted code runs so it cannot touch anything else.

YOUR NOTES · PART I

PART II

The solution at a glance

The core idea, the life of one finding, what is new, and who uses it.

9 The core idea

RAKSHA wraps an AI model inside a frame of hard, checkable tools. Three kinds of work happen in order: **find** (gather evidence a weakness exists), **fix** (propose changes, cheapest first), **prove** (judge each change with checks that cannot be argued with). Only proven results reach a person.

10 A finding's life story

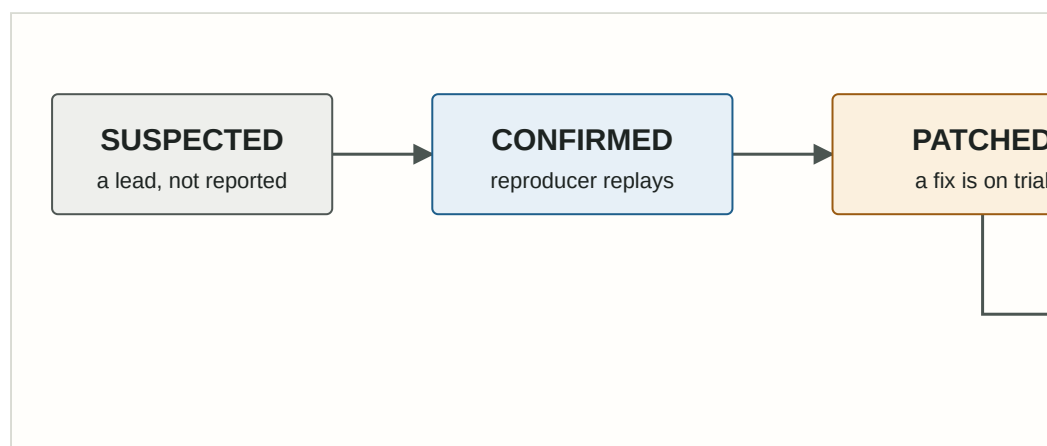


Figure 1 · Every weakness moves through these states. Only CONFIRMED and later are ever reported.

1. **Suspected:** a lane notices something. It stays private.
2. **Confirmed:** the reproducer makes it happen again, observed by an oracle (Chapter 17).
3. **Patched:** a candidate fix is applied to a scratch copy and put on trial.

4. **Verified:** the fix passed all five gate checks (Chapter 21). If no candidate passes within three rounds, the finding is **report only:** proven, but referred to a human.

11 What makes RAKSHA different

IDEA	WHAT IT MEANS	WHY IT MATTERS
Proof-carrying findings	Every report carries a replayable reproducer and a signed bundle	A commander acts on evidence, not opinion
Five-check gate	A fix is judged by five independent checks, never by the model	No plausible-but-wrong fixes ship
Rules of engagement for autonomy	Autonomy is a per-system authority level, like fire control	Humans keep command on critical systems
Build-free first	Useful proven findings even when code will not compile	Something real always comes out
It says no, in public	Shallow, unsafe and weak fixes are refused on screen	Refusal is what makes the yes believable
Vaccine sweep and memory	One proven fix becomes a fleet-wide check and a reusable pattern	The estate gets safer faster
Offline by construction	An automated guard and live counters prove nothing leaves	Fits sealed Army networks

12 Who uses it

PERSON	WHAT THEY NEED	WHERE THEY GET IT
Operator	Run it, watch progress, approve or reject fixes	Console: Mission Board, Live Pipeline, Finding Detail
Developer / project owner	What is wrong in my code, what changed, what must I do	Projects screen, versioned reports, needs-a-human list
Commander	Mission risk, readiness, decisions in plain language	Commander's Brief (English/Hindi), mission view, certificate
Auditor	Can every claim be re-checked?	Evidence Vault, signed reports, compliance pack
Jury	Does it really work, offline, honestly?	Demo beats, Scorecard, verifier media on their own laptop

KEY TAKEAWAYS

- Find, fix, prove: in that order, with proof at every step.
- Seven ideas set RAKSHA apart; all are built.
- Five kinds of people use it, each with their own view.

YOUR NOTES · PART II

The journey, end to end

One example codebase followed through every stage of the pipeline: what happens inside, how it moves to the next stage, and what you see on the console meanwhile.

W The example

A unit hands over a small project called **signals-parser** on a USB stick. It contains:

- `src/tlv.c` : a C function that reads incoming messages. It copies each message into a fixed 16-byte buffer without checking the length (a real weakness: a longer message overwrites memory next to the buffer).
- `config/app.properties` : a database password written in plain text.
- `requirements.txt` : a Python helper that pins `pyyaml==5.3.1`, a version with a published advisory.

The asset registry lists signals-parser as **mission-critical**, mission function "message parsing", owner unit "Signals". The finale box has a C compiler and no model server running (so we can see the AI-free path).

IN PLAIN WORDS

Three different problems in one project: a memory bug that only shows up when the code runs, a leaked password, and an old library. Watch how each is handled differently, and what a person sees at each step.

The timeline at a glance

TIME	STAGE	STATUS OF FINDINGS	WHAT YOU SEE
0 s	1 · Media accepted	—	"accepted"; the time-to-first-finding clock starts
≈1 s	2 · Staged as data	—	"staged 14 files"
≈1 s	3 · Project identified	—	project ID, tier mission-critical
≈4 s	4–5 · Build-free lanes	password, library: CONFIRMED	amber card, first finding, clock stops
≈5 s	6–7 · Entry point, harness	—	"harness synthesized for parse_record"
≈6 s	8–10 · Fuzzing, alarm, confirm	memory bug: SUSPECTED → CONFIRMED	"CWE-121 confirmed at src/tlv.c:11"
≈7 s	11 · Repair ladder, hygiene	PATCHED	candidate on trial
≈7–12 s	12–13 · Gate, extra proofs	VERIFIED	five green ticks
≈16 s	14 · Red team	VERIFIED (held)	"red team held"
≈17 s	15–17 · Authority, evidence, report v1	3 items need a human	Approve buttons, brief, signed report
later	18 · Owner acts, run again	report v2	green ✓ fixed lines, certificate

Times are typical of the demo machine for a project this small; a large codebase takes minutes to hours, with the same stages.

1 Media accepted

You do: on the console's Demo Beats screen, type the USB path (for example `/media/usb/signals-parser`) and press **Ingest**. (At the finale, targets placed in `/targets` are picked up the same way at start.)

Inside: the intake desk checks the path is a real folder under an allowed media root, and that no other intake is running. If either check fails you get a one-line reason and nothing else happens.

You see: "accepted", and a clock "time to first finding: 0.3 s and counting".

Moves on when: the checks pass; a background worker takes over so the console never freezes.

2 Staged as data

Inside: every file is copied into a private scratch folder, as plain data. Shortcuts and special files are skipped and counted. Execute permissions are dropped, so nothing on the stick can be run by accident. Limits stop a huge or hostile medium (file count, total size, size per file, folder depth); if a limit is hit the copy stops there and the record says "truncated".

You see: event log line "signals-parser: staged 14 files (9,812 bytes); 0 links skipped".

3 Project identified

Inside: RAKSHA looks in its project store. No project called signals-parser with this file layout exists, so it mints `PRJ-signals-parser-...`. It looks the name up in the asset registry: mission-critical, Signals, "message parsing". That tier caps the rules of engagement at **R1 (recommend)**: RAKSHA may prove and draft fixes, but applies nothing.

You see: nothing yet on the board; the ID appears with the first report (Stage 17).

4 Build-free lanes run

Inside: every file is read once, by every lane at the same time:

- **Secrets lane** matches `db.password=...` in `config/app.properties`. It records the rule, file and line, and a masked preview (only the first and last characters). Its reproducer is a re-check command: run the same detector on the same line again.
- **Supply lane** parses `requirements.txt`, finds `pyyaml 5.3.1`, and checks the offline advisory database: it is inside a known affected range, fixed in 5.4. It

also checks whether the code actually imports it (reachability) and prepares a version-bump patch.

- **Crypto, config, API and binary lanes** find nothing here.

Because each match can be re-run exactly, these two findings go straight to **CONFIRMED**.

You see: a new **amber** card on the Mission Board, "signals-parser · build-free mode, still finding". Event log: "build-free scan complete (2 findings, 2 proven)" and "first finding on the board 3.9 s after the media went in". The clock stops.

5 Findings annotated and ranked

Inside: duplicates from different lanes are merged. Each finding gets its asset tier, a CVSS 3.1 score, ATT&CK and D3FEND labels, and a place in the risk register (severity × mission tier × known-exploited data).

You see: Live Pipeline shows two findings in the CONFIRMED column; the risk register lists them in fix-first order.

6 Deep lane: finding an entry point

Inside: RAKSHA reads the C source (never running it yet) looking for functions that take input, here `parse_record(const char *data, int len)`. It scores candidates by shape and name. With a model server running, the model would also help rank them; here the deterministic ranking is used.

You see: "signals-parser: looking for an entry point to fuzz (no harness shipped)".

7 Harness written and checked

Inside: RAKSHA writes a small C driver that feeds bytes to `parse_record`, using random file and symbol names for this run. It compiles the driver with the target code and the memory-error detector, inside the sandbox. A two-part quality check follows: it must compile, and it must run one ordinary input cleanly. A driver that fails is discarded and the next entry point is tried.

You see: "harness synthesized for parse_record (c/c++); quality gate passed".

8 Fuzzing in the sandbox

Inside: the driver starts once and then forks a fresh child process for each input, so each test is isolated but fast. Inputs are generated by small changes to a seed set (flip a byte, grow, shrink, repeat, insert boundary values). Each child has no network and is cut off after 2 seconds if it hangs. The search stops when a crash appears or the budget is spent.

You see: Scorecard execution count climbing; the target card stays amber.

9 The alarm rings

Inside: one input longer than 16 bytes makes the copy write past the buffer. AddressSanitizer, the memory alarm compiled into the build, stops the child and prints exactly what happened and where: *stack-buffer-overflow in parse_record, src/tlv.c line 11*. The finding is born as **SUSPECTED**.

10 Confirming it is real

Inside: before anything is reported:

1. The input is replayed in a fresh one-shot run; the alarm must ring again (twice).
2. **Attribution:** if the crash were inside RAKSHA's own driver, it would be dropped. It is in `tlv.c`, the target's code.
3. **Minimise:** the input is shrunk to the smallest one that still rings the alarm, so the evidence is easy to read.
4. **Contract check:** if the function stated a precondition no real caller can break, it would stay suspected with the reason. Not the case here.

Status becomes **CONFIRMED**, and the reproducer is stored with a fingerprint.

You see: event log, highlighted: "signals-parser: CWE-121 confirmed at src/tlv.c:11; reproducer of 17 bytes replays". The finding appears in Finding Detail.

11 The repair ladder and patch hygiene

Inside:

1. **Template:** the "bound the copy to the buffer size" template matches this shape and produces a two-line change.
2. **Fix memory:** checked for a past proven fix of the same kind (none yet). With a model server, a model candidate would come next, only if cheaper rungs fail.
3. **Hygiene:** the candidate touches only `src/tlv.c` (the fix site), adds 2 lines (limit 150), and adds no command, network or code-loading call. Admitted.

Status becomes **PATCHED** (on trial).

You see: the Live Pipeline card moves to PATCHED.

12 The five-check gate

Inside: two scratch copies are built in the sandbox, the original and the patched.

CHECK	WHAT HAPPENS	HERE
1 COMPILES	Both copies build	pass
2 POV_DEAD	The 17-byte reproducer runs on the patched copy; the alarm must stay silent	pass
3 DIFFERENTIAL_CORPUS	Normal inputs run three times on the original (unstable ones set aside), then on both copies; outputs must match exactly; the project's own tests must pass; it must not be much slower	pass: 5 inputs identical, tests pass
4 COVERAGE_HELD	The patched line must still be reached by normal inputs (the feature was fixed, not deleted)	pass
5 CLEAN_REFUZZ	24 close variations of the reproducer and a fresh fuzzing pass on the patched copy; the same weakness must not return, and no new kind of crash may appear	pass

You see: in Finding Detail, the gate strip fills with five green ticks, one by one. Had any check failed, it would turn red with the reason, and the ladder would try the next candidate (up to three rounds).

13 Extra proofs

Inside: the fix is re-checked on an optimised release build; the Z3 solver proves the new bound always holds; undoing the patch is shown to restore the exact original bytes. Each proof is added to the record as extra evidence.

14 The red team

Inside: an independent round tries to show the fix does not hold, with its own wider set of test inputs (including many input lengths) for a few seconds. It finds nothing. Had it succeeded, the record would show the fix as broken.

You see: "independent red team held; no input broke it (277 attempts)". Status: **VERIFIED**.

15 Authority: who may apply it

Inside: signals-parser is mission-critical, so the fix is **recommend only**: it needs two human approvals and is never applied by RAKSHA.

You see: in Finding Detail, the split-screen replay (original: alarm rings; patched: silent), the two-line patch, the gate record, the red-team result, and **Approve** / **Reject** buttons with "2 signatures required". The Commander's Brief tab has the same finding as a short staff paper, in English or हिन्दी.

16 Evidence sealed, memory updated

Inside: an evidence bundle is written and signed for each of the three findings. Every step above is in the hash-chained journal. The proven fix is stored in fix memory, and a vaccine sweep checks other codebases for the same mistake.

You see: Evidence Vault lists the three; pressing **Verify** re-checks every byte. The Scorecard updates precision, fixes, timings and model calls (0).

17 Report v1

Inside: the first signed project report is written:

- **Needs a human (3):** rotate the database password (with a checklist); approve the prepared pyyaml upgrade (it needs a build); approve the proven memory fix (recommend-only asset).
- Posture score, exposure clocks started, the secret recorded as exposed, the library inventory and attack surface noted.

You see: Projects screen: signals-parser · v1 · score · 3 need you. One-page summaries in English and Hindi. The readiness certificate is refused while a critical item is open.

18 The owner acts, and RAKSHA checks

The owner: applies the approved patch, rotates the password and removes it from the file, bumps pyyaml to 5.4, and marks the items done. Then runs signals-parser again.

Inside: the same pipeline runs. The secrets and supply lanes no longer match, which is itself proof. The memory bug is not re-found. Report v2 is written, chained to v1, and the marked-done items are re-checked rather than trusted.

You see: report v2: ✓ fixed password, ✓ fixed pyyaml, ? not re-found memory bug (shown honestly: a crash not seen again is not counted as proven fixed unless its old reproducer is replayed), "confirmed closed by this run" for each marked item, the trend line, and now a readiness certificate.

19 Learning, quietly

The two reports are added to the learning history: one memory fault, one secret, one dependency, owner unit Signals. If the same family keeps recurring across projects, a draft guideline appears on the Learning screen for ACG to approve. From now on, the pre-merge check can stop the same mistake before it ships.

When things go differently

- **No crash found in the budget:** the card stays amber; the build-free findings stand; the record says the deep lane found nothing (never "secure").
- **No fix passes the gate:** after three rounds the finding is REPORT ONLY, with the reason for each refusal, and goes on the needs-a-human list.
- **The red team breaks a fix:** the record shows it broken; the fix is not offered for approval.
- **A toolchain is missing:** that language's deep lane is skipped and says so; build-free lanes still run.
- **With a model server:** stages 6 and 11 can call the model for ranking and fix candidates; everything after it is unchanged, because the gate decides.

KEY TAKEAWAYS

- Fast proven findings in seconds, deep ones as the fuzzing finds them.
- Every stage leaves a visible trace on the console and in the journal.
- Nothing is applied without the authority the asset allows, and every claim can be replayed.

YOUR NOTES · THE JOURNEY

The architecture, layer by layer

How code goes in, how weaknesses are found and proven, how fixes are made and judged, and how everything is kept sealed and recorded.

13 The big picture

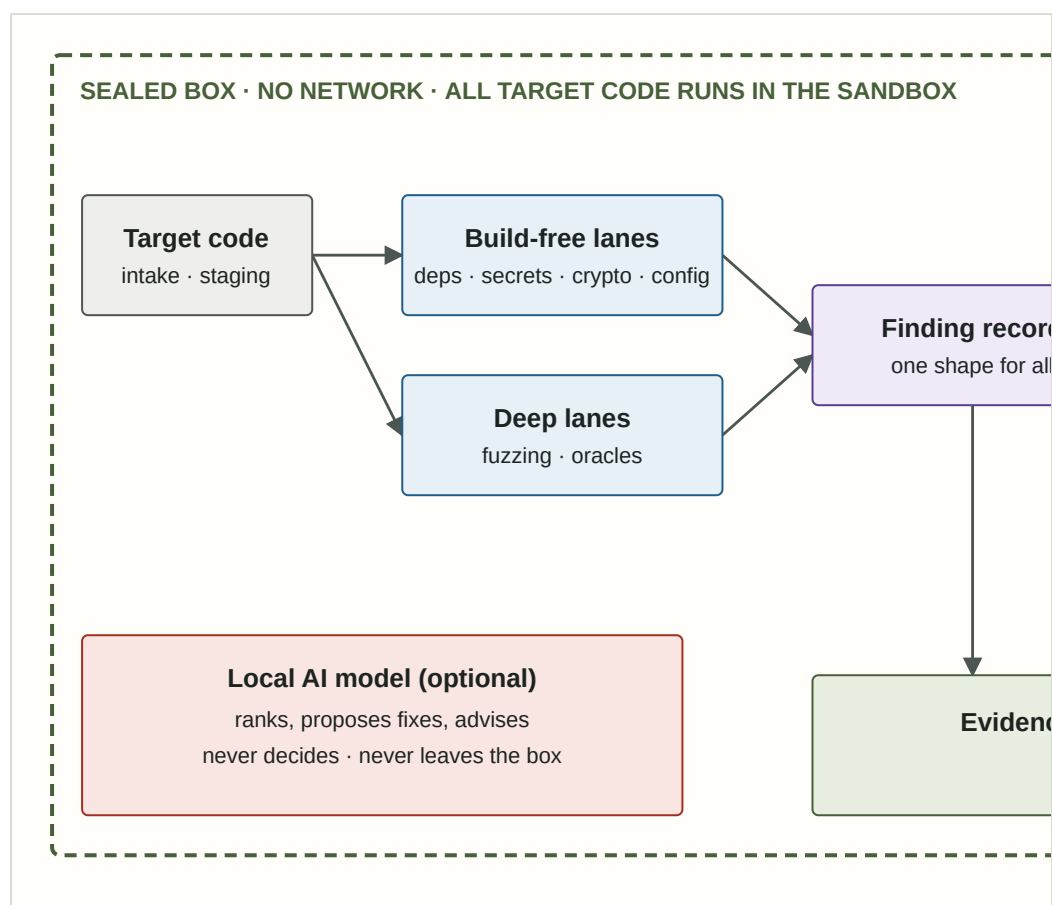


Figure 2 · The whole system runs on one sealed machine. Blue finds, purple records, amber fixes, green judges, olive stores and shows.

FOR ENGINEERS

Python 3.11, standard library only at run time; optional extras (the SMT solver) degrade to a documented fallback when absent. The orchestrator (`raksha/orchestrator.py`) holds the live session; the console is a stdlib HTTP server (`console/server.py`). Every spawn of target code goes through one door, `raksha/sandbox.py` .

14 Getting code in

Code arrives three ways: mounted at start (the finale's `/targets` folder), ingested on the console from a judge's USB media (**bring your own target**), or scanned from the command line.

BUILT · SAFE INTAKE

- Only paths under allowed media folders are accepted.
- The tree is copied as plain data into a private scratch folder, within limits (files, total size, size per file, folder depth).
- Shortcuts (symlinks), devices and other special files are skipped and counted, never followed.
- Scanning happens off to the side and is merged into the live board in one step, so the console never freezes or shows half a result.
- Time to first finding is measured and shown live (4 seconds on the demo estate).

TERM EXPLAINED

Symlink: a file that points to another file elsewhere. A hostile medium could use one to trick a tool into reading outside the folder, so RAKSHA refuses them.

15 Build-free lanes: checks that need no compiling

IN PLAIN WORDS

Many real weaknesses can be proven just by reading files: an old library with a known flaw, a password left in a config file, encryption turned off. These checks work on any language, need no build, and take seconds.

LANE	WHAT IT PROVES	EXAMPLE
Supply chain	A pinned dependency version is inside a known advisory's affected range; whether the code actually imports it	log4j-core 2.14.1 (Log4Shell)
Secrets	A credential is written in a file	a cloud access key in <code>app.properties</code>
Crypto	Weak or outdated cryptography; certificate checks turned off	MD5 for signatures, TLS 1.0
Config / infrastructure	Risky settings in configuration files	debug mode on in production config
API specs	Endpoints with no authorisation, debug endpoints exposed	<code>DELETE /users/{id}</code> with no security
Binaries	Library versions read from compiled files; a software bill of materials	a vulnerable library inside a <code>.jar</code>
Git history	A secret that was committed once and later deleted	a key in an old commit
External scanners (optional)	OSV-Scanner, Gitleaks, Checkov confirmed by re-match; Semgrep leads confirmed only by RAKSHA's own lanes	a misconfigured Dockerfile
Structure	A map of the code (who calls what) used to rank risk	input reaching a dangerous call

TERMS EXPLAINED

Dependency: a library your code borrows. **Advisory**: a public notice that a version of a library has a flaw. RAKSHA carries an offline copy of the OSV advisory database. **SBOM** (software bill of materials): an ingredients list of the software.

Each build-free finding has a **deterministic re-check**: `python -m raksha secret-match ...` and similar commands re-run the same detector on the file, so a judge can replay it.

16 Deep lanes: running the code to find real crashes

IN PLAIN WORDS

RAKSHA finds the functions in the code that take input, writes a small test driver for each, and feeds it thousands of varied inputs inside the sandbox. When one input makes the program misbehave, that input becomes the reproducer.

TERMS EXPLAINED

Fuzzing: automatically trying huge numbers of varied inputs to make a program fail. **Harness**: the small driver that feeds inputs to the code under test. **Synthesized harness**: one RAKSHA writes itself, so targets that ship no test driver can still be tested.

LANGUAGE	HOW IT IS TESTED	NEEDS
C / C++	Synthesized harness, memory-error detector, fast fork-server loop	gcc or clang
Python	Synthesized harness; dangerous calls watched (e.g. a shell built from input)	Python
Java	Synthesized driver for public entry points, one JVM for many inputs	JDK (Maven for project tests)
JavaScript / TypeScript	Node harness with dangerous-call guard	Node
Go	Go's own built-in fuzzing	Go toolchain
Rust	Cargo-based fuzzing; panics caught	Rust toolchain

Each input runs in a short-lived child process, cut off after two seconds if it hangs, so the search never stalls. Crashes are re-checked twice, shrunk to a minimal input, and attributed: a crash inside RAKSHA's own driver is dropped, never reported.

PARTIAL · DEPENDS ON TOOLCHAINS

A language's deep lane runs only if its toolchain is installed. The current sealed image carries gcc, JDK and Maven; Node, Go, Rust and clang are a team step (Chapter 54).

17 Oracles: the alarm bells

IN PLAIN WORDS

An oracle is an alarm that rings only when something genuinely bad happens, such as a write past the end of a buffer. Because the alarm is observed, the finding is a fact, not a guess.

ORACLE	RINGS WHEN
AddressSanitizer (ASan)	memory is read or written out of bounds, or used after it was freed
UndefinedBehaviorSanitizer (UBSan)	arithmetic overflows or other undefined operations
LeakSanitizer (LSan)	memory is never released
ThreadSanitizer (TSan)	two threads race on shared data
Hang detector	an input makes the program stop responding
Python / Node sink guards	untrusted input reaches a shell or code-execution call (recorded and blocked, never executed)
Java (Jazzer-style) detector	a known dangerous Java behaviour such as JNDI lookups
Go / Rust panic	the program crashes with a panic
Behaviour baseline	an input makes the program touch files, sockets or processes it never touches normally
Metamorphic checks	two inputs that should give the same answer do not

TERM EXPLAINED

CWE (Common Weakness Enumeration): a standard numbering of weakness types, e.g. CWE-121 stack overflow, CWE-78 command injection. Each oracle labels what it saw with a CWE.

18 The finding record

Every lane, in every language, produces the same record. That is why one gate, one console and one report format serve everything.

FOR ENGINEERS · KEY FIELDS

oracle , bug_class (CWE), language , target , severity , frames (stack), fix_site_set (ranked places to fix), reproducer (bytes + replay command + hash), replay_before/after , gate (five results), patch_diff , repair_lane , red_team , roe_level , plus proofs (bound, reachability, rollback). Status changes are guarded: a record cannot jump to VERIFIED without the gate history behind it. Source: raksha/finding.py .

19 The repair ladder

Fixers are tried cheapest first. Each candidate goes to the gate; the first that passes all five checks wins (or, with the frontier on, the best of several that pass).

1. **Templates** (no AI): known fix shapes, e.g. "bound this copy to the buffer size", "pass arguments as a list instead of a shell string".
2. **Fix memory** (no AI): a fix proven earlier for the same kind of weakness, re-applied; it can even carry a fix idea across languages.
3. **AI model**: proposes a patch, optionally with a test; if the gate refuses it, the model gets the gate's reason and one more try.
4. **Mitigation**: a blunt but safe floor, labelled as such.

At most three rounds. If nothing passes, the finding is **report only** and the record says exactly which check refused each attempt.

BUILT · MEASURED

On the demo set, 100% of verified fixes so far cost zero AI calls (templates and memory). The mutation factory shows templates fix 8 of 8 disguised variants of known bugs (it was 6 of 8 until the local-model benchmark exposed a template gap, now closed).

20 Patch hygiene: refusing unsafe fixes before they are tried

IN PLAIN WORDS

Before a fix is even built, three rules check it. A fix may only touch the files where the weakness is; it must stay small (at most 150 added lines); and it may not introduce anything that runs commands, opens network connections or loads code, unless the original already did. A fix that breaks a rule is refused and the refusal is recorded.

This matters because an AI model reads the target's code, and that code could contain text designed to steer the model. Hygiene stops a steered fix before it is ever run.

21 The five-check gate

IN PLAIN WORDS

The gate is a strict examiner. It asks five questions about a fix. Only a fix that passes all five is called verified.

#	CHECK	THE QUESTION	CATCHES
1	COMPILES	Does the original build, and does the patched one?	broken fixes
2	POV_DEAD	Does the original reproducer no longer trigger the alarm?	fixes that do not fix
3	DIFFERENTIAL_CORPUS	Do normal inputs give exactly the same output as before? Do the project's own tests pass? Did it get much slower?	fixes that break normal behaviour
4	COVERAGE_HELD	Is the fixed code still reached by normal inputs?	"fixes" that just delete the feature
5	CLEAN_REFUZZ	Do close variations of the reproducer, and a fresh fuzzing pass, still trigger the same weakness?	shallow fixes that only block the one known input

TERMS EXPLAINED

POV (proof of vulnerability): the reproducer. **Corpus**: a collection of normal test inputs. **Coverage**: which lines of code an input actually runs.

Differential: comparing before and after on the same inputs.

The gate fails closed: if a target cannot run a check (for example, no fuzzing possible), the fix is not verified and the record says why. A fix that passes all checks on too little evidence is also held back as report only.

22 Extra proofs

- **Deployment twin**: the fix is re-checked on an optimised release build, not only the instrumented test build.
- **Bound proof**: for size-limit fixes, a mathematical solver (Z3) proves the new limit always holds.
- **Reachability proof**: the solver shows the bad state was reachable before the fix and is not after.
- **Rollback proof**: undoing the fix returns the code to exactly its original bytes.
- **Regression test**: a test is kept only if it fails before the fix and passes after.

TERM EXPLAINED

SMT solver: a program that proves or disproves mathematical statements about code automatically. These proofs are extra evidence; they never replace the gate.

23 The red team

IN PLAIN WORDS

After the gate says yes, an independent checker tries to show the fix still does not hold, using a wider and differently chosen set of test inputs than the gate used. If it succeeds, the fix is marked broken on the record. The red team can only ever take a "yes" away; it can never give one.

The "it says no" demo beat shows this live: a weak fix passes the gate, and the red team catches it.

24 The AI models

WHERE THE MODEL HELPS	WHERE IT IS KEPT OUT
Ranking which functions to test first	Deciding whether a weakness is real (oracles do)
Proposing repair candidates and tests	Deciding whether a fix works (the gate does)
Advisor, judge and red-team roles whose disagreements are shown	Anything leaving the box (sealed mode refuses remote endpoints)
Similarity search over past fixes	Severity scores (computed from the record)

PARTIAL · MODEL SERVER

RAKSHA talks to any OpenAI-compatible model server. The finale design serves a ~32B coding model with vLLM on the box's GPU. No model is bundled in the repository; without one, RAKSHA runs model-free and still finds and proves, fixing only the shapes its templates and memory know.

BUILT · BENCHMARKED WITH A LOCAL MODEL

Four runs with Qwen2.5-Coder-7B on CPU (docs/model-benchmark.md) found six weaknesses in the model path, all now fixed: patch hygiene refused the standard command-injection fix; small models write malformed diffs (now they may send "search / replace" edits and RAKSHA writes the exact diff); the prompt contradicted the fix; the model server returned one candidate instead of three; empty patches reached the gate; and a template gap meant two variants were never fixed. The 7B model itself added no fixes; the finale design uses a ~32B model on the GPU.

TERMS EXPLAINED

LLM (large language model): an AI that writes text and code. vLLM: software that serves a model on a GPU. Sealed mode: a setting that refuses any model endpoint that is not on the box itself.

25 The evidence bundle

Every reportable finding gets a sealed folder: the proof record, the replay script and reproducer, the patch and rollback script, the regression test, plain-language report, Commander's Brief, environment details, a provenance statement and a signature over all of it.

IN PLAIN WORDS

A bundle is a receipt. Change one byte anywhere in it and verification fails. A judge can check it on the console's Evidence Vault, or on their own laptop with the verifier media.

BUILT · SIGNING

Bundles are signed with HMAC-SHA256 using a deployment key the installer generates on the box (never shipped, never committed); without one, the published demo key is used and every bundle says so. When a cosign key pair is provisioned, every bundle also gets a public-key seal that anyone can verify but no one without the private key can forge. Post-quantum signatures are added when the library is present.

26 The session journal

Every event is appended to a diary in which each line carries a fingerprint of the line before it. Change or delete any line and the chain breaks. The journal lets a crashed run resume exactly, powers the live event log, and feeds the time-lapse that replays a whole run minute by minute.

TERM EXPLAINED

Hash chain: a list where each entry includes a fingerprint (hash) of the previous one, so tampering anywhere is detectable.

27 Sandbox and air gap

- **One door:** every launch of target code goes through `raksha/sandbox.py` ; a test forbids any other route.
- **No network, fewer powers:** the sandbox container has no network and drops system capabilities; a setting refuses to run target code at all without it.
- **Air-gap guard:** an automated check proves RAKSHA's own code contains no network client outside the single local model connector, and the console loads nothing from outside.
- **Live egress counter:** the console shows packets sent on every network interface and RAKSHA's own outbound calls. Pull the cable: the counter goes flat.
- **Per-run names:** files RAKSHA places in a target use random names each run, so target code cannot detect and dodge the tests.

28 Rules of engagement and the asset registry

LEVEL	NAME	MAY
R0	Observe	scan and map only
R1	Recommend	test in sandbox, prove, draft and gate fixes; apply nothing
R2	Act with approval	stage a verified fix; apply after human sign-off
R3	Act autonomously	apply verified fixes within standing orders

The **asset registry** (`raksha/data/assets.json`) names each system, its mission function, owner unit and tier. The tier caps the level: mission-critical systems are recommend-only with two approvers. Risky patches (touching authentication or crypto) step down automatically. Unknown systems are treated cautiously.

29 Risk and scoring

- **CVSS 3.1** score computed exactly from the finding; CVSS 4.0 vector recorded (its score table is not bundled, so no score is invented).
- **MITRE ATT&CK and D3FEND** labels: the attacker technique a weakness enables and the matching defence.

- **Mission impact:** severity weighted by the asset's tier.
- **Attack graph:** how findings chain across the estate; the cheapest path is highlighted.
- **Calibration:** each lane's trust is measured against ground truth, with confidence intervals.

TERMS EXPLAINED

CVSS: the standard 0–10 severity score for weaknesses. ATT&CK: MITRE's catalogue of attacker techniques. D3FEND: MITRE's matching catalogue of defences.

30 Post-quantum readiness

Future quantum computers will break today's public-key cryptography. RAKSHA inventories every cryptographic primitive in the estate (where RSA, elliptic curves and weak hashes are used) and produces a migration plan to quantum-safe algorithms, shown on its own console screen.

KEY TAKEAWAYS

- Two kinds of finding (read-only and run-the-code), one record, one gate.
- The gate and the red team decide; the model only proposes.
- Everything is sealed, signed, chained and replayable.

YOUR NOTES · PART III

Project memory and learning

Every project gets an identity and a signed, versioned history, so its owner can see their security any time, and the system learns which faults keep coming back.

31 Project identity

Each project gets one ID for life, like `PRJ-mixed-estate-36273DF0`. It is recognised by its name and the shape of its files, not by where it sits on disk and not by file contents, so a patch never changes the ID and scanning from a different folder finds the same project. A different project with the same name gets its own ID.

BUILT

`raksha/projects.py`. Everything is stored on the box under `RAKSHA_HOME` (default `~/ .raksha`).

32 Versioned reports

Each run writes the next report version: v1, v2, v3. Every version is signed and chained to the one before, like the journal. Nobody can quietly drop a finding between v3 and v4, edit an old report, or remove one; verification would fail for every later version.

33 The colour-coded "what changed" report

MARK	MEANING	RULE
 fixed	Gone since last version	Only when the absence is itself a proof (a deterministic re-check no longer matches)
 ? not re-found	A crash not seen this run	Shown, but never counted as fixed
 + new	Not in the last version	Includes anything a patch introduced
 • still open	In both versions	With how many versions it has been open
 needs a human	RAKSHA may not or cannot close it	With the reason and a checklist

Colour never appears alone: every line carries an icon and a word, so the report works for colour-blind readers and on a black-and-white printout.

34 Needs a human

REASON	WHY A PERSON
Rotate a credential	Only its owner can revoke and reissue it
Policy decision	Who may call an endpoint is the system owner's call
Dependency upgrade	A prepared version bump needs a real build to verify
Crypto migration	Changing algorithms needs a key and data plan
Gate could not certify	Proven weakness, no fix passed all five checks
Recommend-only asset	The fix is proven, but rules of engagement require approval

Each item comes with a checklist. When someone marks it done, RAKSHA does not take their word for it: the next run re-checks, and the report says "confirmed closed" or "still present".

35 Score, deadlines, exposure-days, secrets, certificate

- **Posture score** (0–100): 100 minus weighted open weaknesses (severity × asset tier). The breakdown always adds up exactly. Below the evidence floor it shows "n/a", never a guess.
- **Deadlines**: each severity has a fix deadline (critical 7 days, high 30, ...), tightened for mission-critical systems. **Exposure-days** counts how long each serious weakness has stayed open.
- **Secrets tracker**: every leaked credential, whether it was rotated, and for how long it was exposed.
- **Data-handling statement**: what was read, and the measured proof that nothing left the box during the run.
- **Attack-surface drift**: a warning when a new version adds a network listener, file upload or privileged call, even before any bug is found.
- **Readiness certificate**: a signed statement ("as of v7: no open critical weakness, all fixes proven, 3 items await a human"). Issued only when the record supports it.

36 Self-learning

IN PLAIN WORDS

RAKSHA studies its own history across projects. It counts which kinds of fault keep recurring, which fixed faults came back, and which unfamiliar libraries appear. From that it proposes guidelines for ACG and learns new things to watch for, but nothing it learns takes effect without proof or approval.

- **Recurring-fault analytics**: by fault family, language, component and owner unit, with trends and regressions.
- **Guideline proposals for ACG**: a fault family seen across two or more projects becomes a draft rule with evidence counts, anonymised examples and the proven fix pattern. It changes nothing until a named person approves it.
- **Learning new technology**: unfamiliar libraries are logged; a lesson (for example, "treat this call as privileged") starts on probation.

37 Keeping learning safe

- Evidence counts only when it comes from a gate-verified record.
- A lesson on probation applies only inside the project that proposed it. It is promoted only after three verified cases from at least two different projects.
- One false alarm demotes a lesson at once, and the project that taught it is quarantined.
- Every lesson keeps its full history, and any lesson can be rolled back with one click; the test suite proves rollback restores prior behaviour.

38 Pre-merge check and compliance pack

Pre-merge check (`python -m raksha.premerge PATH`): developers run it before merging a change. It blocks on an approved guideline, on the return of a previously fixed weakness, or on anything critical. **Compliance pack**: a quarter's signed reports and certificates bundled as verifier media, so an auditor re-checks every signature and the chain on their own machine.

NOT BUILT · FOR THE TEAM

Three Wave 5 items are left to the team: an explainer of how a weakness could be reached (K21), answering questions about a project from its reports (K24), and a process drill on a copy of a project (K30). Generating them was stopped by an automated safety classifier during the build. Their specifications are in `PLAN_V2.md`, and the code they plug into exists.

KEY TAKEAWAYS

- Every project has an ID and a tamper-proof, versioned history.
- The report shows what changed, colour-coded, and what a human must do.
- Learning proposes; proof and people decide.

YOUR NOTES · PART IV

PART V

The operator console (UI and UX)

One offline web page served from the box. Fourteen screens, English and Hindi, readable on a projector.

39 Design principles

- **Offline:** one self-contained page; no font, script or image from outside. The air-gap guard checks it.
- **Honest:** every value comes from the record. If something was not measured, the screen shows a dash.
- **Glanceable:** state is shown by colour, icon and word together.
- **Two voices:** each finding can be read as a staff-officer summary or as engineer detail.
- **Bilingual:** one key switches English and Hindi (हिन्दी).
- **Projector-ready:** a high-contrast mode with heavier lines and larger text.

40 Screen-by-screen tour

1 • Mission Board

Every target as a card: green (built and deep-tested), amber (build-free mode, still finding), red (could not read). Counts by status and the rules-of-engagement level per target.

2 • Live Pipeline

Findings flowing through suspected → confirmed → patched → verified, plus the ranked risk register: fix these first.

3 • Event Log

The run's story in plain sentences: "harness synthesized for parse_record", "candidate 2 rejected at CLEAN_REFUZZ", "VERIFIED; fix proven (template lane)".

4 • Finding Detail (the main screen)

Search, filter and sort; the five-check gate strip; split-screen replay (vulnerable build fires, patched build silent); the patch; red-team result; proofs; staff and engineer views; operator actions.

5 • Attack Graph

Findings as a graph of steps across the estate, with the cheapest path lit and each edge explained on hover.

6 • Estate Map

Targets by asset tier against status, and the top three risks.

7 • Post-Quantum

The cryptography inventory and the migration table: file, line, primitive, replacement.

8 • Evidence Vault

Pick any reportable finding and press Verify: every hash and the signature are re-checked live.

9 • Scorecard

The jury's measures, live: precision, findings, fixes, speed, model calls and tokens, cloud calls (0), sandbox status, what the run did not establish.

10 • Commander's Brief

A JSSD-style brief per finding, in English or Hindi, exportable as PDF (English).

11 • Demo Beats

Run the "it says no" sequence; ingest a judge's media with a live time-to-first-finding clock; the egress counter for the cable pull.

12 • Time-lapse

A slider over a recorded run's journal: findings, targets and what the system was doing at any moment.

13 • Projects

Every project's latest score and open items; per project the colour-coded "what changed", needs-a-human checklists with Mark done, version picker, English/Hindi one-page summary, readiness certificate, and developer / commander / auditor views; the mission view and posture over versions.

14 • Learning

Draft guidelines for ACG with Approve / Reject (a name is required), lessons with Roll back, recurring-fault analytics and unfamiliar libraries.

41 Operator actions

Approve, reject, mark false positive, re-run the red team, export a bundle, issue an advisory, run the demo beat, ingest media, zero the egress counter, mark a needs-human item done, issue a certificate, decide a guideline, roll back a lesson. Every action is recorded in the journal with who did it and why. Actions need a per-session token that only the served page holds, so a stray page on the machine cannot trigger them.

42 Accessibility

- Keyboard: number keys switch screens, **/** searches, **j** / **k** move through findings, **h** high contrast, **e** language, **?** help.
- Colour always paired with icon and word.
- Light and dark themes; works on a phone width.

43 What people take away

OUTPUT	FOR	FORM
Evidence bundle	Engineers, auditors	Signed folder, replay script
Commander's Brief	Commanders	On screen (English/Hindi), PDF (English)
Findings export	Staff	CSV, Excel
SARIF	Other security tools	Standard JSON format
Internal-CERT advisory	A CERT	Signed, numbered RAKSHA-CERT-YYYY-NNNN
Project report and one-page summary	Project owners	Signed, versioned; HTML, phone and print friendly
Readiness certificate, compliance pack	Approval files, auditors	Signed documents, verifier media
Verifier media	Judges	A folder (USB) that re-checks everything with only Python

TERM EXPLAINED

SARIF: a standard file format for sharing static-analysis results between tools.

CERT: a computer emergency response team, which issues security advisories.

YOUR NOTES · PART V

Trust and honesty

How every claim is backed, what has been measured, and where the limits are.

44 How claims are proven

- **Tests:** 873 automated tests, run with every slow language lane on; 0 failures at the last run.
- **Benchmark:** `python -m raksha.benchmark` regenerates every number in the benchmark report from a real run.
- **Negative controls:** a deliberately clean estate scanned after every change; any finding there is a false alarm.
- **Baseline comparison:** the same targets through a standard static scanner and through RAKSHA.
- **Recall:** measured where the number of real bugs is known.

45 Measured results so far

READ WITH CARE

These come from RAKSHA's own small, curated demo set. They are a reproducible baseline, not a statistical claim about real-world code.

MEASURE	RESULT	SET
Reports carrying a replaying reproducer	100%	by construction
False alarms on clean controls	0 of 11	negative-control estate
Static scanner: reports / with reproducer / defect files hit	8 / 0 / 5	7 demo targets
RAKSHA: reports / with reproducer / defect files hit	7 / 7 / 7	7 demo targets
Recall, mutation factory	100% (8/8)	disguised variants of known bugs
Fix rate, mutation factory	100% (8/8)	same variants, zero model calls
Local 7B model (CPU) added fixes	0	4 runs; exposed 6 pipeline weaknesses, all fixed
Extra findings from wired-in scanners	90 in ≈ 15 s	OSV-Scanner, Semgrep, Checkov, Gitleaks on the demo estate
Recall, fuzzable demo ground truth	54.5% (6/11)	misses named; some need toolchains absent here
Verified fixes needing no AI call	100%	demo set
"It says no" beat, all four verdicts as scripted	≈ 25 s	C demo target
Verifier media full check	≈ 0.3 s	demo media
Intake time to first finding	≈ 4 s	mixed-language estate

46 The threat model: how RAKSHA protects itself

RAKSHA reads untrusted code, so it treats every target as potentially hostile. Sixteen attack surfaces are listed in `docs/threat-model.md`, each with its control and test. Highlights: the sandbox door; refusing links and special files at intake; patch hygiene against steered fixes; replay scripts that quote every argument; random per-run names; sealed mode for the model; signed and chained records.

47 The claim audit

`docs/claim-audit.md` maps every claim made to the jury to the code and test behind it, or rewords the claim. Nothing in the brief claims more than the build does.

48 Honest limitations

LIMITATIONS

- No measured results yet on large, real-world codebases (the real-ARVO benchmark run is not built here).
- Deep testing depends on finding an entry point that takes input; unusual code may yield no crash in the time budget.
- Without a local model, automatic fixes are limited to known shapes.
- The 36-hour endurance runs and the model server under load have not been run on the finale hardware.
- Public-key (cosign) seals are optional and need a key pair provisioned at install.
- The Hindi Commander's Brief is on screen and in text, not in the PDF export.
- K21, K24 and K30 are not built.

YOUR NOTES · PART VI

Deployment and the finale

The sealed box, how it is installed, how the demo runs, and how to prepare.

49 The sealed box

- **Hardware:** one machine with a 32 GB GPU for the model; a CPU-only fallback profile exists.
- **Containers:** the RAKSHA image (orchestrator and console), the sandbox image (where target code runs, no network), and the model server (vLLM) on an internal network with no route out. The console is published only to the machine itself.
- **Install:** `deploy/install.sh` from the sealed SSD with the cable out: verify the bundle checksums, load the carried images (no download), confirm the sandbox is present, start the stack.

50 Profiles

PROFILE	FOR	EFFECT
Datacenter	The finale GPU box	Full budgets
Node	A smaller server	Reduced budgets
Edge	A laptop or field machine	Smallest budgets, slices still run
CPU-only	No GPU available	Model on CPU or model-free

51 The demo arc

1. The problem, in one line.
2. **Pull the cable:** egress counter flat, cloud calls 0, and the system keeps running.
3. Mission Board lights up; amber means "build-free mode, still finding".
4. A famous weakness (Log4Shell) is found and proven live.
5. Split-screen proof: vulnerable build fires, patched build silent.
6. **It says no:** three bad fixes refused on screen, the good one accepted.
7. Vaccine sweep: one fix, the same mistake found in other systems.
8. Rules of engagement: a critical system's fix now waits for two signatures.
9. Commander's Brief in Hindi; a project's v1 → v2 report; the judge's own media ingested live.

52 Rehearsals on the finale hardware

Run the whole system on the actual finale machine before the event: install from the SSD with the cable out; two long runs (up to 36 hours) on unseen code to catch slow failures such as a filling disk, creeping memory or a stalled model server; practise the three scripted beats with the presenter; record a backup video on two devices. Steps are in `docs/rehearsal-runbook.md`.

53 Fallback ladder

1. Model server down: continue model-free (templates, memory) and say so.
2. A language toolchain missing: that lane goes amber; build-free still runs.
3. Console problem: the command-line slices print the same results.
4. Live demo fails: narrate over the recorded backup.

54 Needed from the team

ITEM	WHY	STATUS
Serve a coding model on the box (vLLM) and set the endpoint	Broader automatic fixes	to do
Add Node, Go, Rust and clang to the images	Deep lanes for those languages	to do
Generate a deployment signing key at install	Bundles not on the demo key	built into install.sh
Optional: cosign key pair, OPA on, scanner lanes on	Public-key seals, policy check, extra evidence	switch on
Fill the asset registry with the jury's systems	Correct tiers and approvals	on the day
Two long runs, three beats, backup recording	Proven on the real box	to do
Build K21, K24, K30 (optional)	Complete Wave 5	not built

YOUR NOTES · PART VII

Present state and ideal state

Where we are now, wave by wave, and what the finished system should look like.

55 What is built, wave by wave

WAVE	THEME	STATUS
0	Foundations: event journal and resume, crash signatures, weakness families, profiles, build flavours	built
1	Finding and proving: multi-bug campaigns, new oracles, supply chain, Java driver, binaries, behaviour baseline, rollback proof, threat model	built
2	Reasoning: solver proofs, guided model output, cross-language fix memory, model roles, calibration, CVSS and ATT&CK	built
3	The console: 10 screens, operator actions, Hindi, exports, high contrast	built
4	Jury and product: "it says no", intake, egress counter, verifier media, advisory, time-lapse, baseline, recall, claim audit	built · J3 reworded
5	Project memory and learning: IDs, versioned reports, needs-a-human, score, certificate, learning, pre-merge, compliance pack, Projects and Learning screens	built · K21/K24/K30 team

56 What is left

- **Team steps** (Chapter 54): model server, toolchains, signing key, asset registry, rehearsals.

- **Not built:** real-ARVO benchmark runner (J3); K21, K24, K30.
- **Never run here:** the long endurance runs and model under load on finale hardware.

57 Present state vs ideal state

Present state (today)

A complete, tested system that runs offline on one machine. It finds and proves weaknesses in six languages plus any language build-free, fixes known shapes without AI, judges every fix with five checks, signs everything, keeps a versioned history per project and shows it all on a 14-screen bilingual console. It has been demonstrated on RAKSHA's own demo targets, without a model server, toolchain-complete images or a run on the finale hardware.

Ideal state (finished)

A sealed box that, installed in twenty minutes with the cable out, ingests any estate the jury hands over, deep-tests every language with full toolchains, fixes most proven weaknesses with a local model under the gate, signs with deployment keys, survives 36 hours unattended, and gives every project owner, commander and auditor a live, versioned, colour-coded picture of their security that they can verify themselves.

AREA	PRESENT	IDEAL
AI model	Wired and benchmarked with a local 7B model on CPU (too slow and weak to add fixes; benchmark fixed six pipeline weaknesses)	Local ~32B coding model on the GPU, measured fix rate on real code
Languages (deep)	C/C++, Python, Java in the image; JS, Go, Rust need toolchains	All six in the sealed images, plus more where demand exists
Real-world evidence	Demo-set baseline, recall on known sets	Published win/loss table on real public bug benchmarks, run on a prep machine
Fix coverage without AI	A few template shapes + fix memory	A larger template library grown from approved guidelines and proven fixes
Signing	HMAC with a key generated at install; optional cosign public-key seal	cosign on by default; post-quantum signatures bundled
Endurance	Harness and watchdog built, short runs only	Two clean 36-hour runs on the finale box
Project memory	Versioned signed reports, learning on probation	Months of history across the estate driving approved ACG guidelines
Owner features	Projects and Learning screens; K21/K24/K30 not built	All of Wave 5, including questions answered from the record
Integration	Pre-merge command	Hooked into every team's build pipeline, alerts to owners
Assets	Demo registry	Every real system registered with tier, owner and mission

How the finished system should look and feel

- **For the operator:** drop in media, watch the board fill, approve fixes with evidence in front of them.
- **For a project owner:** open their project any time and see v1 → v12, what changed, and the short list of what is theirs to do.
- **For a commander:** a one-page Hindi or English summary, mission impact, deadlines, and a readiness certificate for the approval file.

- **For ACG:** a quarterly pack of proposed guidelines backed by evidence, adopted with a signature.
- **For an auditor or judge:** a USB stick that re-checks every claim on their own laptop in under a minute.

58 Future ideas

- A fast classifier model (for example a decision model such as Jev) for triage and labelling, advisory only, where an on-premise version exists.
- Continuous mode across the estate, re-checking every project when the advisory database updates.
- More languages and frameworks learned under probation.
- Spin-offs: a CERT advisory service, a developer training tool built from proven fixes, a supplier-code acceptance check.

59 Your suggestions

OVERALL

FEATURES TO ADD OR CHANGE

FOR THE DEMO AND THE JURY

APPENDICES

Reference

A · Glossary

TERM	MEANING
Advisory	A public notice that a library version has a flaw
Air gap	Physically disconnected from all outside networks
ASan / UBSan / LSan / TSan	Sanitizers that detect memory errors / undefined behaviour / leaks / data races
Attack graph	How weaknesses could chain from one system to another
Build-free	A check that reads files without compiling or running anything
Bundle	A signed folder of evidence for one finding
Corpus	A set of normal test inputs
Coverage	Which lines of code an input actually runs
CVSS	The standard 0–10 severity score
CWE	A standard number for a type of weakness
Dependency	A library your code uses
Deterministic	Same input, same answer, every time
Differential check	Comparing old and new behaviour on the same inputs
Egress	Data leaving the machine over a network
Fuzzing	Trying many varied inputs to make a program fail
Gate	The five checks a fix must pass
Harness	A small driver that feeds inputs to code under test
Hash / hash chain	A fingerprint of data / a list where each entry fingerprints the last
HMAC	A signature made with a shared secret key
LLM	Large language model, an AI that writes text and code
Oracle	An alarm that rings only on a real failure
Patch	A change to code that fixes a weakness
POV / reproducer	The exact input that makes a weakness happen
Red team	An independent check that tries to show a fix does not hold

TERM	MEANING
ROE	Rules of engagement: what RAKSHA may do on each system
Sandbox	A sealed container with no network where untrusted code runs
SARIF	A standard file format for security findings
SBOM	Software bill of materials, an ingredients list
SMT solver (Z3)	A program that proves mathematical facts about code
Template	A known fix shape applied without AI
vLLM	Software that serves an AI model on a GPU

B · Project map

WHERE	WHAT
<code>raksha/orchestrator.py</code>	The live session; entry point of the deployment
<code>raksha/finding.py</code>	The finding record and its states
<code>raksha/lanes/</code>	Build-free lanes
<code>raksha/harness/</code> , <code>raksha/adapters/</code>	Deep lanes, synthesized harnesses, language adapters
<code>raksha/oracles/</code>	The alarm bells
<code>raksha/autorepair.py</code> , <code>repair.py</code> , <code>hygiene.py</code>	Repair ladder and patch hygiene
<code>raksha/gate/</code>	The five-check gate
<code>raksha/redteam.py</code>	The red team
<code>raksha/bundle.py</code> , <code>journal.py</code>	Evidence bundles and the session journal
<code>raksha/sandbox.py</code> , <code>airgap.py</code>	Sandbox door, air-gap guard, egress counter
<code>raksha/projects.py</code> , <code>reports.py</code> , <code>learning.py</code> , <code>views.py</code>	Wave 5: identity, reports, learning, views
<code>console/</code>	The operator console
<code>deploy/</code>	Images, compose files, install script
<code>docs/</code> , <code>RAKSHA_AI_Dossier/</code>	Reports, runbook, threat model, claim audit, dossier

C · Commands

```
python -m raksha.orchestrator      # serve the console on :80
python -m raksha.saysno             # the "it says no" beat
python -m raksha.benchmark          # regenerate the benchmark
python -m raksha.baseline --report  # static scanner vs RAKSHA
python -m raksha.recall --report    # recall where it is known
python -m raksha.premerge PATH      # pre-merge check
python -m raksha.verifiermedia demo OUT # build judge media
```

```
python -m raksha.timelapse show FILE    # replay a recorded run
python -m raksha.airgap                 # air-gap guard
python -m pytest                         # the test suite
```

D · Settings

SETTING	WHAT IT DOES
<code>RAKSHA_INFERENCE_BASE_URL</code>	Where the model server is
<code>RAKSHA_SEALED</code>	1 (default) refuses any model not on the box
<code>RAKSHA_REPAIR_MODEL</code> , <code>RAKSHA_ADVISOR_MODEL</code>	Which models to use
<code>RAKSHA_SANDBOX_IMAGE</code> , <code>RAKSHA_REQUIRE_SANDBOX</code>	The sandbox image; refuse to run target code without it
<code>RAKSHA_BUNDLE_KEY_FILE</code>	Deployment signing key
<code>RAKSHA_HOME</code>	Where project reports and learning are stored
<code>RAKSHA_INTAKE_ROOTS</code>	Which media folders intake accepts
<code>RAKSHA_TARGETS</code> , <code>RAKSHA_OUT</code>	Where targets are read and results written

E · Evidence index

- `docs/benchmark-report.md` : results, baseline, recall
- `docs/claim-audit.md` : every claim mapped
- `docs/threat-model.md` : 16 surfaces and controls
- `docs/rehearsal-runbook.md` : rehearsals and demo beats
- `PLAN_V2.md` : every item, wave by wave, with status
- `tests/` : 873 tests

F · Sources

- AI Kavach problem statement (competition brief, dossier file 01)
- DARPA AI Cyber Challenge (AIXCC), the modern cyber-reasoning reference

- MITRE CWE, ATT&CK, D3FEND; FIRST CVSS; OSV advisory database
- ARVO and AutoPatchBench (external benchmarks, for the ideal state)

G · Open-source tools and repositories

RAKSHA's own code uses only the Python standard library at run time. Everything below is a tool RAKSHA *drives* (runs as a separate program), a data source it carries offline, or something used only while developing. Each row says why it is used and whether it is in use today.

USED TODAY

Invoked by the committed code.

OPTIONAL / PLANNED

Supported seam or part of the full deployment; not required today.

DEVELOPMENT ONLY

Used to build, test or measure; not shipped to the box.

Languages and toolchains

TOOL	REPOSITORY	WHY WE USE IT	STATUS
Python 3.11 (CPython)	python/cpython	RAKSHA itself, and the Python deep lane; standard library only, so the box needs no extra packages	today
GCC	gcc-mirror/gcc	Builds C targets with the memory alarm (AddressSanitizer) and coverage (gcov)	today
LLVM / Clang (sanitizers, libFuzzer)	llvm/llvm-project	The sanitizer alarms (ASan, UBSan, LSan, TSan) and an optional faster fuzzing engine	today (sanitizers) · optional (libFuzzer)
OpenJDK + Apache Maven	openjdk/jdk · apache/maven	The Java deep lane: compile the synthesized driver, run project tests	today
Go	golang/go	Go's built-in fuzzing and coverage for the Go lane	needs toolchain
Rust / Cargo	rust-lang/rust · rust-lang/cargo	The Rust lane: build, fuzz, catch panics	needs toolchain
Node.js	nodejs/node	The JavaScript/TypeScript lane and its dangerous-call guard	needs toolchain
Git, GNU patch	git/git	Applying and reversing patches exactly; the git-history secrets lane	today

Proof, signing and formats

TOOL	REPOSITORY	WHY	STATUS
Z3 SMT solver	Z3Prover/z3	Mathematical proofs that a fix's bound always holds, and reachability before and after	today (optional extra, baked into the image)
SARIF 2.1.0 schema	oasis-tcs/sarif-spec	Standard format so other security tools can read RAKSHA's findings	today
CycloneDX	CycloneDX/specification	Standard software bill of materials from the binary lane	today
in-toto	in-toto/in-toto	Standard provenance statement inside each evidence bundle	today (format) · planned (tooling)
Sigstore cosign	sigstore/cosign	Public-key seal on every evidence bundle: verifiable by anyone with the public key, forgeable by no one without the private key	wired · optional
Open Quantum Safe (liboqs)	open-quantum-safe/liboqs	A post-quantum signature next to the standard one, when the library is present	optional

Models and serving

TOOL	REPOSITORY	WHY	STATUS
vLLM	vllm-project/vllm	Serves the coding model on the box's GPU, offline, with an OpenAI-compatible interface	deployment
llama.cpp	ggml-org/llama.cpp	Runs a model on CPU when no GPU is available	fallback
Qwen coder models	QwenLM/Qwen3-Coder	Open-weight coding model for fix candidates (Apache-2.0)	deployment
Foundation-sec-8B (Cisco)	model weights on Hugging Face	Smaller security-tuned model for the advisor role	planned

Isolation

TOOL	REPOSITORY	WHY	STATUS
Docker (Moby)	moby/moby	The sandbox containers (no network, dropped capabilities) and the sealed deployment	today (when provisioned)
gVisor	google/gvisor	A stronger container sandbox for untrusted code	planned

Fuzzing engines and detectors (full deployment)

TOOL	REPOSITORY	WHY	STATUS
AFL++	AFLplusplus/AFLplusplus	A mature coverage-guided C/C++ fuzzer that plugs into RAKSHA's fuzzer seam	optional
honggfuzz	google/honggfuzz	Alternative C/C++ engine	optional
Jazzer / Jazzer.js	CodeIntelligenceTesting/jazzer	JVM and Node fuzzing with built-in security detectors (the Log4Shell demo uses its detector style)	today (demo API) · optional
Atheris	google/atheris	Coverage-guided Python fuzzing	optional
OSS-Fuzz (PySecSan)	google/oss-fuzz	Origin of the Python security-sanitizer approach RAKSHA extends	today (approach)

Static analysis, dependencies, secrets, config (full deployment)

TOOL	REPOSITORY	WHY	STATUS
OSV-Scanner / OSV data	google/osv-scanner · google/osv.dev	The open advisory database RAKSHA carries offline, plus the scanner as an independent dependency check	today (data) · wired · optional (scanner)
Semgrep	semgrep/semgrep	Static leads on a bundled offline ruleset; they count only when RAKSHA's own lanes confirm the same site	wired · optional
Joern	joernio/joern	A deeper code property graph than RAKSHA's built-in structure lane	planned
Gitleaks	gitleaks/gitleaks	A second opinion for the secrets lane, confirmed by re-match	wired · optional
Checkov	bridgecrewio/checkov	Infrastructure-as-code checks (Dockerfiles, Kubernetes, Terraform, CI files)	wired · optional
Syft	anchore/syft	Software bill of materials from images and binaries	planned
Open Policy Agent	open-policy-agent/opa	Rules of engagement as policy code; with it on, both the policy and the built-in rule must allow a deploy	wired · optional

Reference data (carried offline)

SOURCE	WHERE	WHY
CWE	cwe.mitre.org	Standard names for weakness types
CVSS	first.org/cvss	Standard severity scores
EPSS	first.org/epss	Likelihood a weakness gets exploited, for prioritising
CISA KEV	cisa.gov	Weaknesses known to be exploited in the wild, for prioritising
MITRE ATT&CK	mitre/cti	Attacker technique labels
MITRE D3FEND	d3fend.mitre.org	Matching defensive technique labels

Benchmarks (for the ideal state)

SOURCE	WHY	STATUS
ARVO (reproduced OSS-Fuzz bugs)	Real public bugs with gold patches; the loader seam is built	runner not built
AutoPatchBench (in meta-llama/PurpleLlama)	Scoring automatic patches	planned

Development only

TOOL	REPOSITORY	WHY
pytest	pytest-dev/pytest	The 873-test suite
Playwright + Chromium	microsoft/playwright-python	Driving the console in a headless browser to check every screen; rendering this book's PDF
flawfinder	david-a-wheeler/flawfinder	The static-scanner baseline for C in the comparison table
Bandit	PyCQA/bandit	The static-scanner baseline for Python in the comparison table
pyflakes	PyCQA/pyflakes	Keeps the code free of unused or undefined names

Why some tools are not wired in yet

TOOL	REASON	PATH
vLLM, Qwen, Foundation-sec	They need the finale GPU; RAKSHA already talks to any such server	Start them on the finale box; set the endpoint
gVisor	A container runtime set on the Docker host, not inside RAKSHA	Install runsc on the finale host and set it as the sandbox runtime
AFL++, honggfuzz, Atheris, Jazzer engines	RAKSHA's own engine already runs every lane; these plug into its fuzzer engine seam	Team task, with the toolchain image work
Joern	Large Java service; RAKSHA's built-in structure lane covers the call map it needs today	Add as an optional take lane if deeper data-flow is wanted
Syft, Trivy, Gripe	RAKSHA already writes CycloneDX SBOMs and OSV-Scanner covers dependency advisories	Add Trivy or Gripe as a redundant second dependency opinion if desired

Licences for each are tracked in `THIRD_PARTY.md` ; anything marked "confirm" there must be checked against the exact version before the bundle is handed over.

End of the book.